

A CODE COPILOT

*on
CodeSearchNet*

RAG

Retrieval-augmented generation over a filtered code corpus

LoRA

Parameter-efficient fine-tuning, tested across 16 configurations

Agent

A minimal retrieve → explain → cite pipeline that knows when to abstain

Applied Languages Model · Google & Reichman Tech School

Instructors: Dr. Barak Or, Or Zwebner · Submitted by: Tal Matsil

OVERVIEW

What we'll cover

01

Motivation & the dataset

Why this project, and what CodeSearchNet provides

02

System design

RAG, LoRA, and a small agent — how they fit together

03

The training investigation

16 configurations with surprising finding

04

Results

Retrieval, the agent, and the final base-vs-LoRA verdict

05

Limitations & what's next

Gaps, and where to take this further

Why build this

A day-to-day coding aid that doesn't depend on an external service — and a chance to get hands-on with three techniques at once, instead of just one in isolation.



Given a natural-language query, the tool locates the most relevant function from a large code corpus, optionally uses a LoRA-tuned generator to explain it, and returns an answer grounded in the original code — or abstains when there's no confident match.

01 · DATASET

CodeSearchNet

A collection of code functions labeled with documentation, gathered from open-source GitHub repositories — the Python subset was used.

347,590

code chunks
after filtering

313,620

train
90.2%

17,007

val
4.9%

16,963

test
4.9%

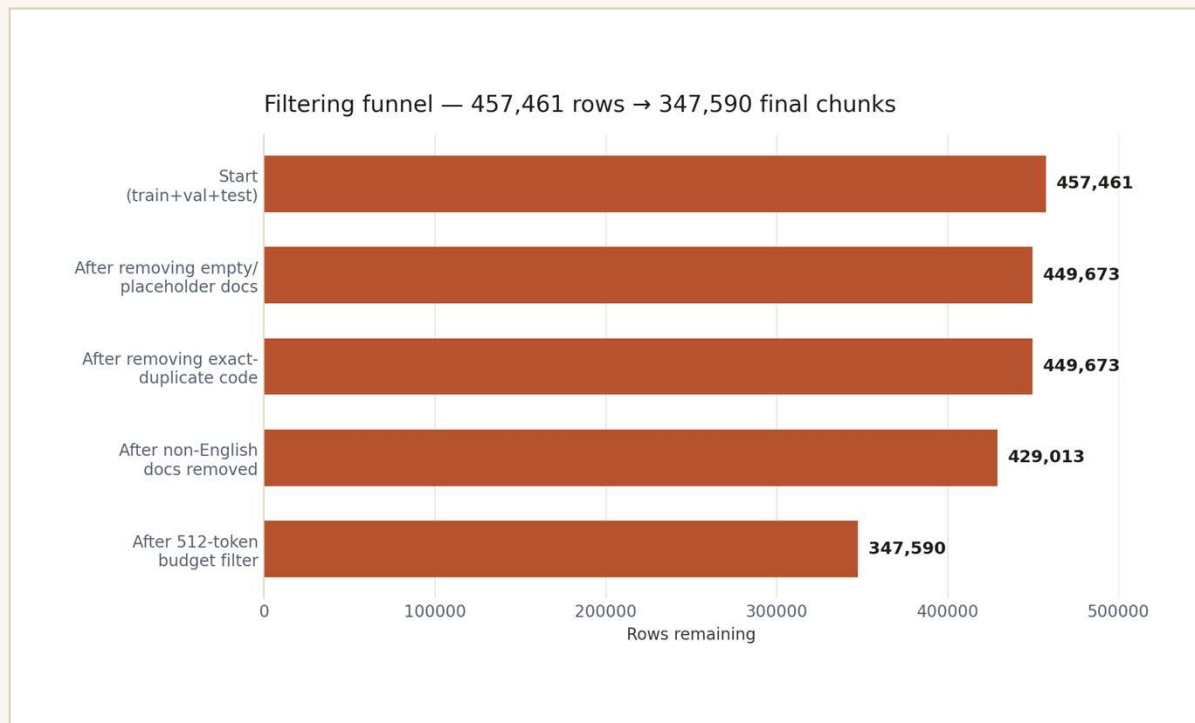


Three quality filters

Empty/duplicate documentation removed · English-language check · 512-token combined budget (doc+code)
Repository-disjoint split: the same code never appears in both train and an evaluation set.

The filtering funnel

Four filters run in order over the full official train+val+test set (457,461 rows), each removing rows unsuited to retrieval or generation.



76% of rows survive

Empty-doc and duplicate filters barely move the bar — rare in real GitHub code. Language filtering removes a modest, real slice. The 512-token budget is the biggest single cut, but three in four rows still reach the final 347,590-chunk table.

Why each filter

Every filtering decision was made for a concrete, practical reason — not applied by default.



Empty / placeholder docs

removed 7,788 rows

A docstring with 2 words or less (e.g. "TODO" or blank) is not a usable retrieval query or a usable generation target. Dropping these protects both training and evaluation from empty signal.



Exact-duplicate code

0 extra rows removed here

When identical code appears in more than one split, keeping it in only one (train over val over test) prevents the same function from being both trained on and used to score the model — which would inflate results.



English-only documentation

removed 20,660 rows

The embedding model and the LLM judges are tuned for English. A docstring in French, Catalan, or another language would be a mismatched query the retrieval and judging pipeline can't fairly score.



512-token budget

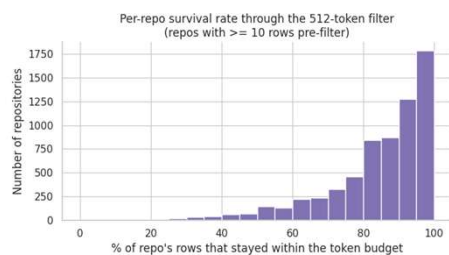
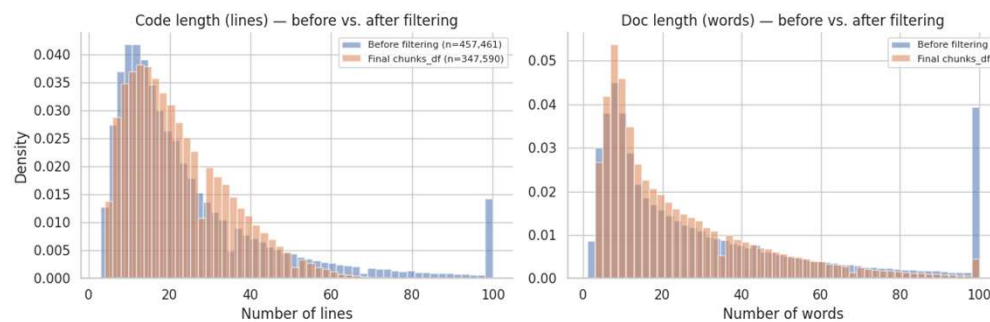
removed 81,423 rows

sentence-transformers/all-MiniLM-L6-v2 — the embedding model used throughout — has a native context length of exactly 512 tokens. Longer rows would be silently truncated at embedding time; better to drop them and measure the loss.

01 • DATASET

Did filtering distort the data?

Comparing the full working set to the final chunk table shows whether filtering only trimmed a tail, or changed the overall picture.



A trimmed tail, not a distortion

Median code length barely moves (18 → 20 lines); median doc length barely moves (19 → 17 words). Across 6,593 repos with 10+ rows, median per-repo survival through the token filter is 88.5% — only 32 repos (~0.5%) lose more than 80% of their rows.

01 - DATASET

A row, before it becomes a chunk

The same real row - proycon/pynlpl, chunk_id=0 - all its raw fields, then the two chunk forms built from it.

BEFORE - raw dataset row (all fields)

repository_name	proycon/pynlpl
func_path_in_repository	pynlpl/formats/foia.py
func_name	AbstractElement.addidsuffix
language	python
func_documentation_string	Appends a suffix to this element's ID, and optionally to all child IDs as well. There is usually no need to call this directly, invoked implicitly by :meth:`copy`
func_code_string	def addidsuffix(self, idsuffix, recursive=True): """Appends a suffix...""" if self.id: self.id += idsuffix ...
func_code_url	github.com/proycon/pynlpl/blob/.../foia.py#L1263-L1271

AFTER - same row, as two chunk forms

chunk_text

for retrieval & display - docstring + full code

Appends a suffix to this element's ID,
optionally to all child IDs as well...

```
def addidsuffix(self, idsuffix, recursive=True):
    """Appends a suffix to this element's ID,
    and optionally to all child IDs..."""
    if self.id: self.id += idsuffix
    if recursive:
        for e in self:
            try:
                e.addidsuffix(idsuffix, recursive)
            except Exception:
                pass
```

func_code_no_doc

for generation - docstring stripped via ast

```
def addidsuffix(self, idsuffix, recursive=True):
    if self.id: self.id += idsuffix
    if recursive:
        for e in self:
            try:
                e.addidsuffix(idsuffix, recursive)
            except Exception:
                pass
```

Two views of every chunk

So the model can't just copy the answer, each row is stored two separate ways.



chunk_text

For retrieval and display

The docstring followed by the full code together — full readability helps make sense of a retrieval result.



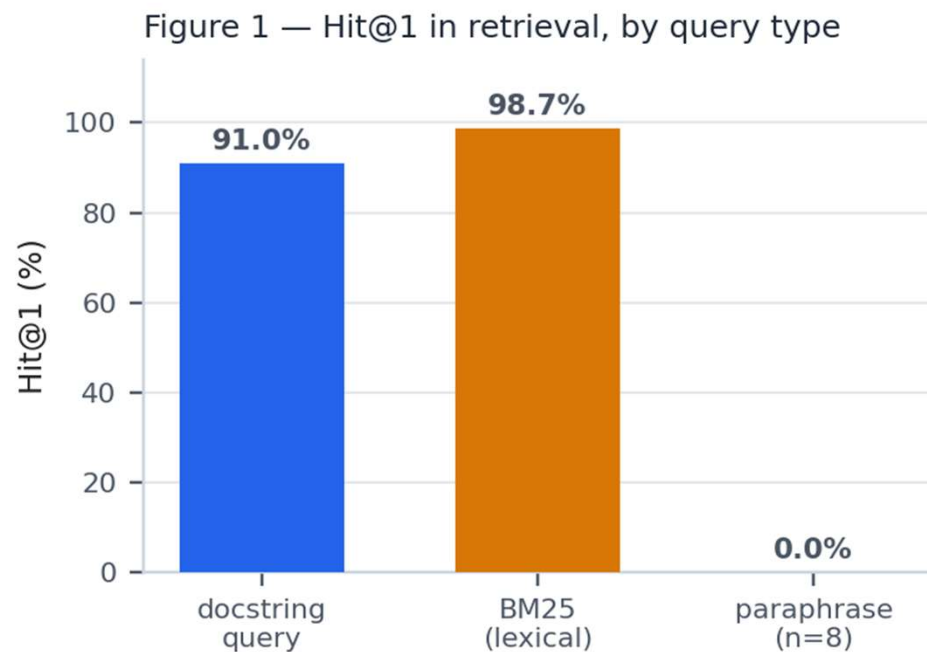
func_code_no_doc

For generation

The same function with its docstring stripped out — the model is forced to produce it, not copy it.

Sanity check for retrieval

Do semantic embeddings add value over plain word overlap? And do docstring queries look like real questions?



Looks good — is misleading

Docstring queries hit 91% Hit@1, but plain BM25 word-overlap scored even higher — 98.7%. When rephrased as real natural-language questions, Hit@1 collapsed to 0%.

Three pieces, working together

01



RAG

Retrieval

Finds the most relevant function from the vector store

02



LoRA

Fine-tuning

A small adapter trained on top of frozen model weights

03



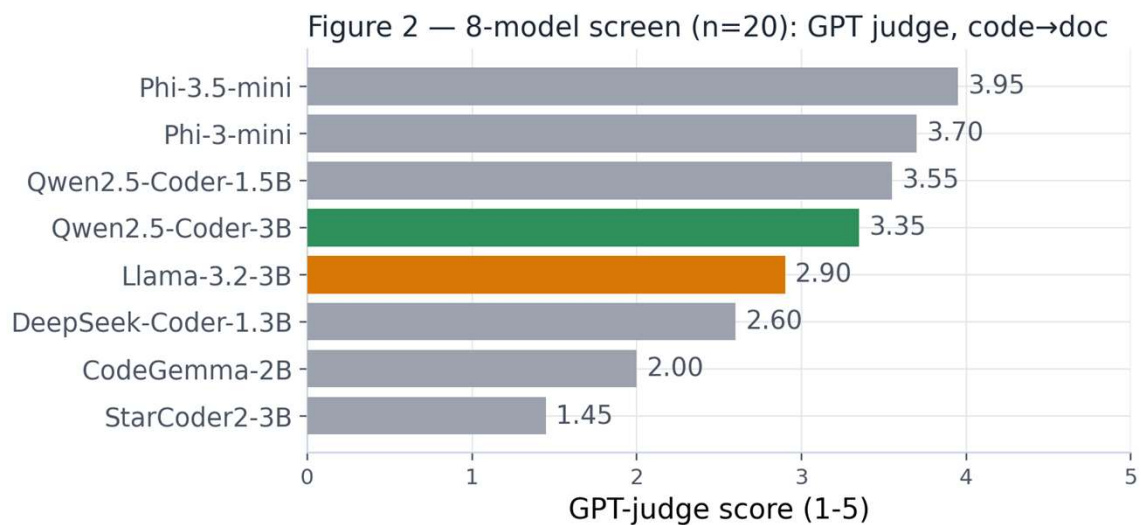
Agent

Orchestration

Retrieve → explain → cite the source, or abstain

Choosing the base model

An early screen of 8 candidates, scored on five weighted metrics



Phi leads on the judge chart alone — but that's one of five metrics.

On the equal-weighted composite:

Llama-3.2-3B
&
Qwen2.5-Coder-3B

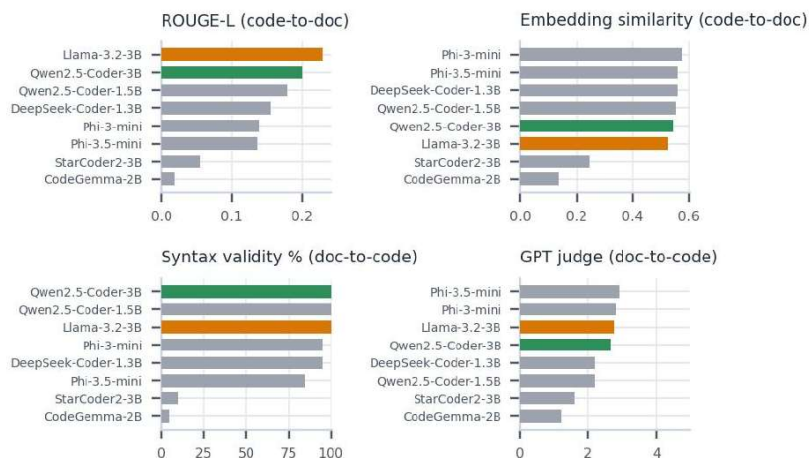
tied at 0.90 each

Qwen wins: ungated + code-specialized training

The other four metrics

GPT-judge score alone favored Phi — but the composite score also weighs ROUGE-L, embedding similarity, and syntax validity.

Figure 3 — remaining metrics in the 8-model screen, n=20 (ROUGE-L, embedding, syntax, judge doc-to-code)



Qwen and Llama, close

Llama leads only on ROUGE-L (word overlap). On embedding similarity, syntax validity, and the doc→code judge score, Llama and Qwen sit close together, with Qwen slightly ahead on two of the three.

Combined with Figure 2's judge score, these five metrics fed the equal-weighted composite score used for the final decision.

A small adapter, a smaller agent

LoRA

Trains only a small set of additional weights on top of a frozen model — a key advantage on a sample of about 400 rows, where full fine-tuning risked memorizing the examples rather than learning from them. Adapters were restricted to attention layers only, to limit the parameter count further.

The Agent

retrieve → **explain** → **cite source**

Kept deliberately small: the critical step is a confidence threshold — if the query is too far from the best retrieval match, the agent answers "I don't know" instead of guessing. A confident wrong answer is worse than an honest admission that there's no answer.

Complexity was deliberately invested in retrieval and tuning — not in a multi-step planning agent.

The surprising finding that drove the research

A first comparison between two LoRA ranks (r=8 vs. r=16) split in an unexpected direction



ROUGE-L rose

Word overlap against the reference docstring improved with the trained adapter.



Judge score fell

Perceived answer quality dropped, despite the higher word overlap.

The model "sounded" closer to the correct answer in wording — but in practice wrote worse answers. That gap prompted an extended test of 16 configurations.

03 · TRAINING INVESTIGATION

16 trained configurations

13 tested systematic variations; 2 tested entirely different questions

Adapter settings & optimization

r = capacity; α = update scale; modules = adapted layers; epochs/LR = training length/step size. Higher = stronger fit — and overfit risk.

Training data construction

Filters set minimum target detail (15+/30+ words); 3× data tests whether cleaner, broader examples generalize better.

10× more training data

Scale test: 10× more examples asks whether the quality drop is mainly a small-data problem.

WHAT EACH METRIC IS FOR · WHAT ITS DIRECTION MEANS

ROUGE-L

Word-sequence overlap with the reference. ↑ = closer wording; may reward imitation.

Gemini judge

Independent second-vendor quality check. Agreement reduces shared-bias risk.

Embedding similarity

Semantic closeness beyond exact words. ↑ = similar meaning, not proof of correctness.

Train loss

Fit on seen examples. ↓ = learning the training set; alone it cannot show generalization.

Full fine-tune vs. LoRA

Full fine-tune updates every weight; LoRA updates only small adapters. LoRA is safer and cheaper on ~400 rows.

Checkpoint & regularization

Checkpoint = saved stopping point; dropout/weight decay = regularizers. More regularization aims to reduce memorization.

Target-rewrite pilot

GPT-4o rewrites target docstrings to test whether noisy or weak labels caused the quality drop.

GPT judge

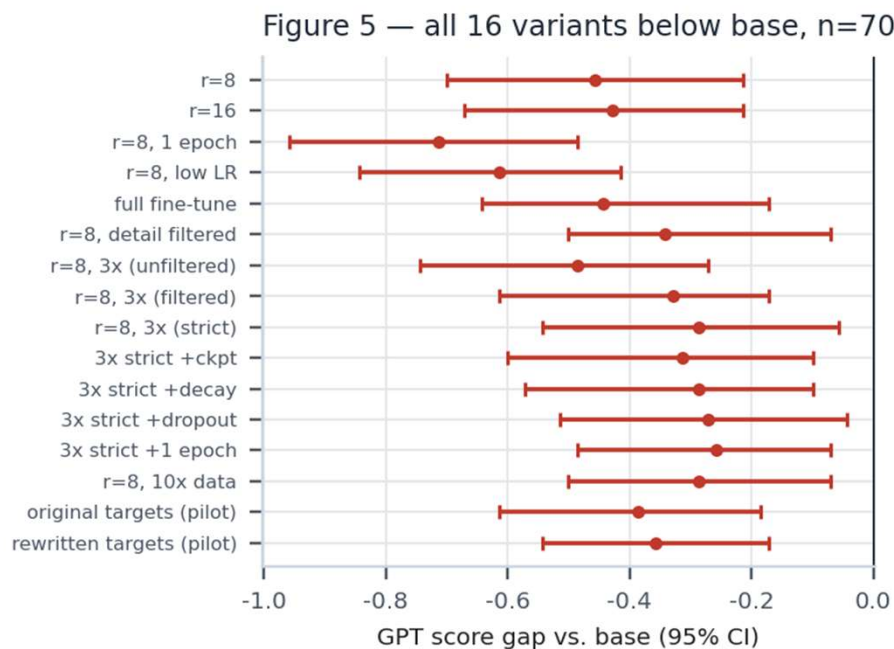
Human-style correctness, relevance, and completeness. ↑ = better answer quality.

Validation loss + gap

Fit on held-out data. ↓ is better; a widening train–val gap = overfitting.

All 16 variants fell short

A bootstrap test (5,000 resamples) confirmed: the drop is statistically real, not noise



Every single configuration

scored significantly below the base model
— including the least-bad version.

LoRA r=8, 3x
strict + 1 epoch

-0.257

gap from base

03 • TRAINING INVESTIGATION

Checked with a second, independent judge

Gemini checked 3 configurations, to rule out bias from a single judge

-0.27

points below base

10× data

-0.49

points below base

original targets

-0.04

points below base

rewritten targets

Two different, independent judges reached the same overall conclusion — the trained model falls below base — though not always with the same intensity.

Final check on the test set

The strongest candidate, checked once more on 40 examples from the TEST split

Metric	Base	LoRA	Wins
ROUGE-L	0.211	0.300	LoRA
Semantic similarity	0.633	0.640	LoRA
GPT judge	3.175	3.050	Base
Gemini judge	4.100	3.850	Base

Automatic metrics favor LoRA — but both judges, meant to reflect human-style quality, favor the base. The same split seen before, now confirmed on unseen data.

04 · RESULTS

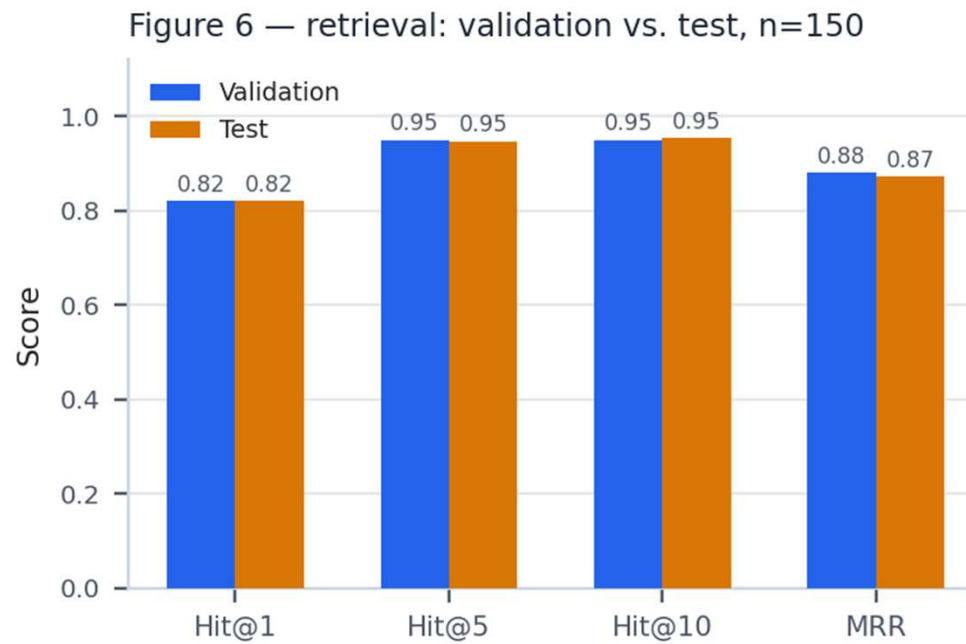
Final check, visualized

The same TEST-split comparison as the table before it — automatic metrics and LLM judges tell opposite stories.



Retrieval: validation vs. test

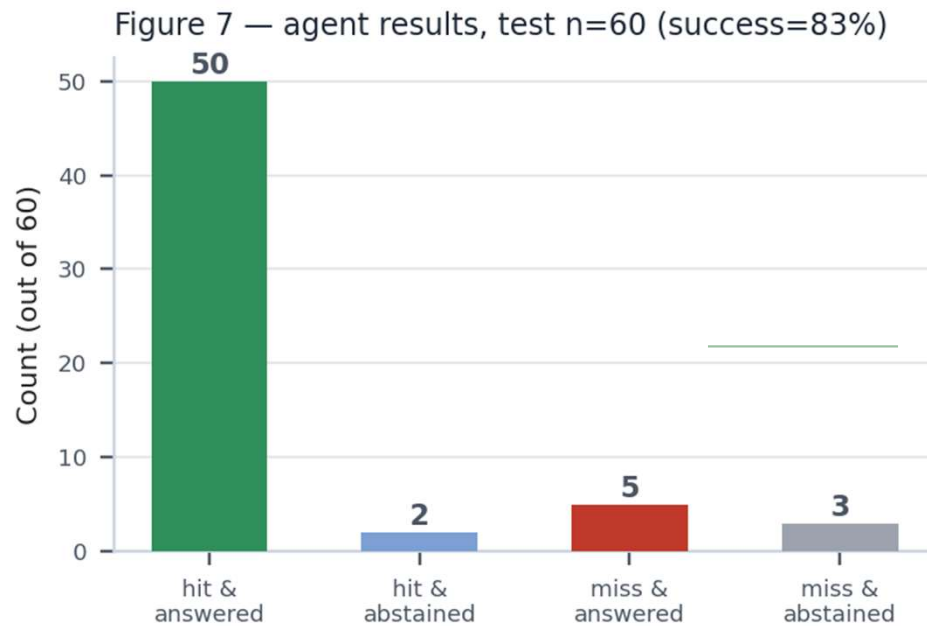
The confidence threshold calibrated on validation holds up well on unseen test data



04 • RESULTS

The agent, end to end

60 test-split questions run through the full agent: retrieve → confidence gate → answer or abstain



83%

success rate

50 of 60: both retrieved the right chunk and answered — slightly above the same check on validation (78%)

The agent, in action



Grounded answer — a real docstring as the query

Question: `"Call IUIAutomationDockPattern::SetDockPosition..."`

Answer (similarity distance 0.184, below the 0.916 threshold):

`"Sets the position of the dock and waits for the operation to complete."`

Source: `yinkaisheng/Python-UIAutomation-for-Windows`



Justified abstention — a natural-language question

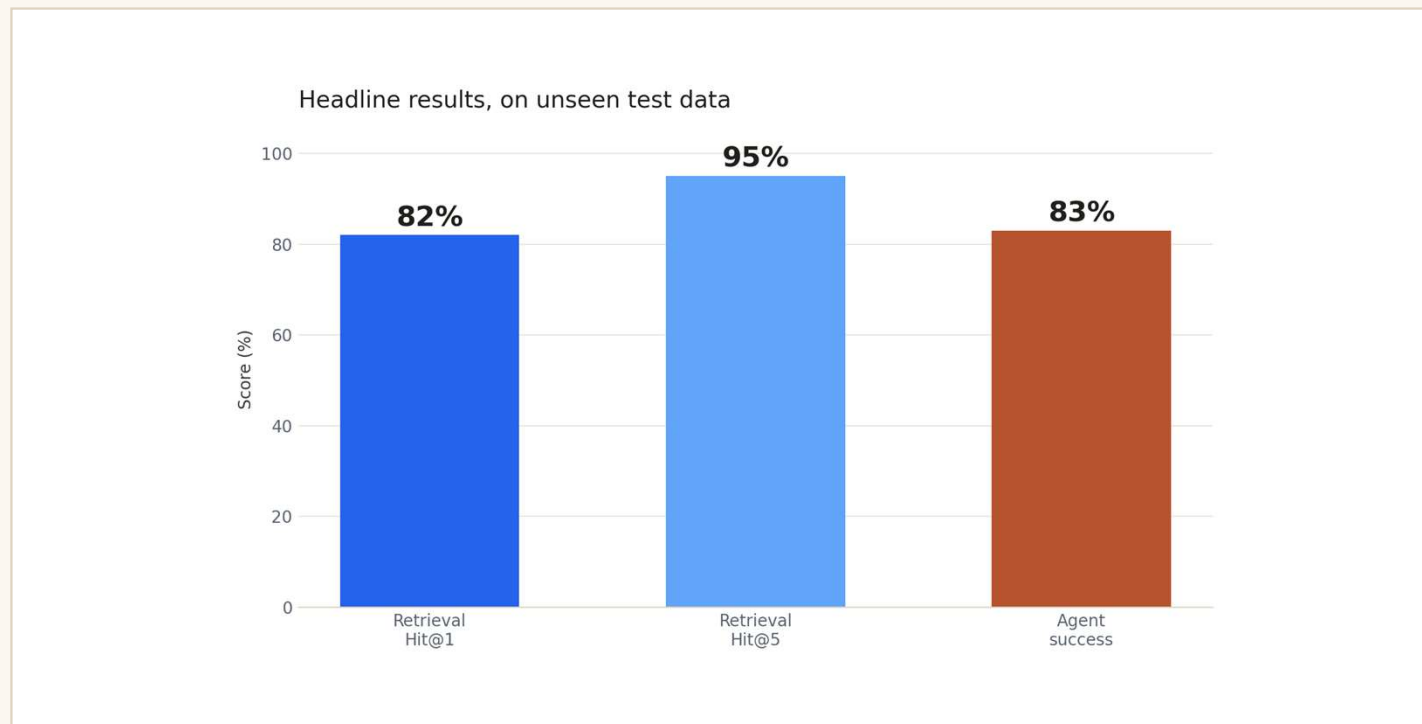
Question: `"How does this repository validate its input?"`

The agent abstained — the closest candidate (distance 1.341) was far beyond the threshold, so it returned "no good enough match" instead of guessing.

04 • RESULTS

The headline numbers

Retrieval and the full agent, both measured on the TEST split the model never trained or calibrated on.



SUMMARY

What this project delivered

82%

Hit@1 retrieval

83%

agent success

16

LoRA configurations

0

beat the base model

The bottom line

Base model Qwen2.5-Coder-3B-Instruct — with no LoRA adapter at all — produced the highest answer quality by two independent LLM judges, while the trained variants improved only on the word-overlap metric (ROUGE-L). The off-the-shelf model was already well suited to the task.

What to read with caution



Docstring vs. natural language

Benchmark queries are closer to code than a real question would be
— real-world performance is likely lower



A single LLM judge's reliability

Two judges agreed on overall direction, but only on 38% of individual answers



Small sample sizes

1,200 training rows, 40 for scoring — out of hundreds of thousands available, due to single-GPU limits



Blanket "I don't know" answers

Even when the base model itself may hold relevant knowledge to answer with

Where this could go next

01



Natural-language questions

Techniques to better handle real questions, not just docstring-style queries

02



Fewer "I don't know" answers

A grounded answer in more scenarios, even when retrieval doesn't find a strong match

03



A bigger rewrite pilot

A larger run of the target-rewrite pilot, and a larger or differently built training set

Thank you



Dataset source

Husain, H., Wu, H.-H., Gazit, T., Allamanis, M., & Brockschmidt, M. (2019)
CodeSearchNet Challenge: Evaluating the State of Semantic Code Search
arXiv:1909.09436